

DOCUMENTACIÓN TÉCNICA

MÓDULO RECOMENDACIONES

Trabajo práctico final – Sistema de gestión de tienda

Instituto Superior de Formación Técnica N.º 179
Tecnicatura Superior en Análisis de Sistemas
Dr. Carlos Pellegrini

ÍNDICE

1. Introducción y objetivo ····· 3
2. Dependencias del módulo ····· 3
3. Clase Popularidad y estructuras ····· 3
4. Carga del estado ····· 3
5. Actualización de popularidad ····· 4
6. Registro y reversión de compras ····· 4
7. Consulta de recomendados ····· 5
8. Integración con transacciones y servidor ····· 5

1. Introducción y objetivo

El módulo BACKEND/recomendaciones.py calcula qué productos son los más populares y permite devolver una lista de recomendaciones. La popularidad se define como la cantidad total de unidades vendidas de cada producto.

Para ordenar los productos utiliza un árbol binario de búsqueda (BST). El módulo mantiene dos representaciones coordinadas: un árbol que permite obtener los productos más populares y un diccionario que registra el conteo actual de cada ID.

2. Dependencias del módulo

Dependencia	Responsabilidad
BD.persistencia	Lee la popularidad histórica agregada desde SQLite mediante leer_popularidad().
BACKEND.tad_arbol	Proporciona insertar_nodo(), eliminar_nodo() y mayores() para operar sobre el BST.
BACKEND.inventario.consultar_producto	Obtiene los datos actuales de cada producto a partir de su ID.

Dependencias inversas: transacciones.py importa recomendaciones.py para registrar o revertir las ventas; servidor.py importa cargar_popularidad() y consultar_recomendados() para inicializar y exponer la API. recomendaciones.py importa inventario.py, pero inventario.py no importa recomendaciones.py, por lo que no se forma un ciclo entre esos dos módulos.

3. Clase Popularidad y estructuras

3.1 Atributo raiz

Popularidad.raiz contiene la raíz del árbol binario de búsqueda. Cada nodo guarda una tupla (popularidad, id_producto). El árbol ordena primero por popularidad y, si dos productos tienen el mismo conteo, utiliza el ID como segundo criterio.

3.2 Atributo conteo

Popularidad.conteo es un diccionario con la forma {id_producto: popularidad}. Permite conocer rápidamente el conteo anterior de un producto. Ese valor es necesario para eliminar del árbol la clave vieja antes de insertar la clave actualizada.

```
raiz del BST:
           (12, 4)
          /       \
      (5, 2)     (18, 7)

conteo = {2: 5, 4: 12, 7: 18}
```

El árbol contiene solamente productos con al menos una unidad vendida. Los productos con popularidad cero no necesitan un nodo. El diccionario conserva la misma regla: cuando el conteo vuelve a cero, se elimina la entrada.

4. Carga del estado

cargar_popularidad() crea una instancia vacía de Popularidad y llama a persistencia.leer_popularidad(). La consulta de persistencia agrupa los detalles de las transacciones por producto y suma las cantidades vendidas.

Por cada fila recibida, inserta en el BST la tupla (popularidad, id_producto) mediante insertar_nodo() y guarda el conteo en el diccionario. Al finalizar, devuelve el estado reconstruido. Esto permite que la popularidad no dependa de los datos que quedaron en memoria durante una ejecución anterior.

La persistencia es la fuente histórica; el árbol y el diccionario son estructuras derivadas que se reconstruyen al iniciar el servidor.

5. Actualización de popularidad

5.1 _cambiar_popularidad(estado, id_producto, variacion)

Es la operación interna que suma o resta unidades a un producto. Obtiene el conteo actual con estado.conteo.get(id_producto, 0) y calcula el nuevo valor sumando variacion.

1. Si el nuevo conteo es negativo, lanza ValueError: no se pueden revertir más unidades de las que figuran como vendidas.
2. Si el producto tenía una popularidad anterior mayor que cero, elimina del BST la tupla (conteo_anterior, id_producto).
3. Si el nuevo conteo es mayor que cero, inserta la nueva tupla (nuevo_conteo, id_producto) y actualiza el diccionario.
4. Si el nuevo conteo es cero, elimina el ID del diccionario y deja el producto fuera del árbol.

No se modifica una clave directamente dentro del BST. La clave determina la posición del nodo; si cambia, el nodo debe retirarse y volver a insertarse para conservar el orden del árbol.

5.2 _unidades_por_producto(productos)

Recibe los detalles de una compra y agrupa las cantidades por ID. Cada detalle debe contener una cantidad que sea un entero positivo. Si el mismo producto aparece en más de una línea, suma sus cantidades. Devuelve un diccionario {id_producto: unidades_totales}.

```
Entrada: [{ "id_producto": 3, "cantidad": 2 },
         { "id_producto": 3, "cantidad": 1 },
         { "id_producto": 8, "cantidad": 4 }]

Salida: {3: 3, 8: 4}
```

6. Registro y reversión de compras

6.1 registrar_compra(estado, productos)

Convierte los detalles en cantidades agrupadas mediante _unidades_por_producto() y, para cada producto, llama a _cambiar_popularidad() con una variación positiva. Cada unidad comprada incrementa en una unidad la popularidad.

La función debe ejecutarse después de confirmar la compra en SQLite y bajo el bloqueo compartido del servidor. Si el guardado de la compra falla, la popularidad no debe modificarse; de ese modo el árbol no reflejará ventas inexistentes.

6.2 revertir_compra(estado, productos)

Primero agrupa las unidades. Luego realiza una etapa de validación completa: comprueba que cada producto tenga en el diccionario una popularidad suficiente para restar las unidades que se desean revertir. Si alguno no alcanza, lanza ValueError antes de cambiar el árbol.

Una vez validados todos los productos, realiza las restas mediante _cambiar_popularidad(). La validación previa evita que una reversión parcial deje el estado de popularidad inconsistente.

7. Consulta de recomendados

consultar_recomendados(estado, inventario, cantidad=5) devuelve hasta la cantidad solicitada de productos, ordenados desde el más popular hacia el menos popular.

Primero valida que cantidad sea un entero entre 1 y MAXIMO_RECOMENDADOS, cuyo valor es 50. Luego utiliza mayores(estado.raiz, cantidad), que realiza un recorrido inverso inorden: visita primero el subárbol derecho, después el nodo y finalmente el subárbol izquierdo. Como el árbol ordena las tuplas de menor a mayor, este recorrido obtiene primero las claves mayores.

Por cada tupla recuperada, toma el ID y consulta el producto en inventario mediante consultar_producto(). Si el producto ya no existe, lo omite. Si existe, agrega el campo popularidad a la copia devuelta por inventario y lo incorpora a la lista de recomendaciones.

El módulo no devuelve solamente un ID o un conteo: devuelve una copia de los datos actuales del producto más su popularidad. Esto permite que el servidor responda con información lista para la interfaz.

8. Integración con transacciones y servidor

8.1 Relación con transacciones.py

Al registrar una compra, transacciones.py prepara y valida la operación, la guarda en SQLite, sincroniza los productos del inventario, apila la compra y finalmente llama a recomendaciones.registrar_compra(). Así la popularidad se actualiza únicamente después de la confirmación persistente.

Al deshacer la última compra, transacciones.py comprueba que la reversión sea válida, solicita a SQLite que revierta la compra y restaure el stock, sincroniza el inventario, desapila la transacción y llama a recomendaciones.revertir_compra(). De esta manera el conteo refleja las unidades que siguen formando parte del historial.

8.2 Relación con servidor.py

Durante el inicio del servidor se ejecuta cargar_popularidad() para reconstruir el árbol. La ruta GET /api/recomendados recibe opcionalmente el parámetro cantidad; si no se especifica, utiliza 5. El servidor convierte el parámetro a entero, llama a consultar_recomendados() dentro del bloqueo compartido y envía el resultado como JSON.

La ruta GET /api/productos también agrega a cada producto el conteo que obtiene de estado.conteo. Así la pantalla de inventario puede mostrar popularidad junto con los datos habituales del producto, sin guardar ese dato como una columna propia en la tabla productos.

```
SQLite (detalle_transaccion)
        │ leer_popularidad()
        ▼
recomendaciones.cargar_popularidad()
```

```
        |
        ▼
Popularidad: BST + diccionario
        ▲                  |
        | registrar /      | consultar_recomendados
        | revertir compra  ▼
transacciones.py       servidor.py → JSON
```